Prof. Haikel Hichri, Email: hhichri@champlaincollege.qc.ca

Lab 06 Exception handling *(SUMBIT ALL YOUR CODE ON MOODLE)*

1. [10 points] Exception handling

Scenario: The Data Connector System

You are building a system that connects to different data sources (a Database and a Cloud Storage).

- The **Interface** defines the contract.
- The **Abstract Class** provides shared logic.
- The **Concrete Classes** throw specific standard exceptions (IOException, SQLException, RuntimeException).

Part 1: The Architecture

1. **Interface Connector**: Defines a connect() method that throws Exception.
2. **Abstract Class BaseConnector**: Implements Connector and adds a validate(String config) method.
3. **Concrete Class DatabaseConnector**: Throws java.sql.SQLException (a checked exception).
4. **Concrete Class CloudConnector**: Throws java.io.IOException (a checked exception).

Implementation:

1. Interface: Connector

- **Method:** void connect() throws Exception Defines the contract for establishing a connection.

2. Abstract Class: BaseConnector (implements Connector)

- **Attribute:** protected String name Stores the identifier for the connection type.
- **Constructor:**) Initializes the name attribute.
- **Method:** validate() Concrete method that prints a validation message for the current name.
- **Method:** abstract method connect() throws Exception, Forces subclasses to implement specific connection logic.

3. Concrete Class: DatabaseConnector a subclass of BaseConnector

- **Constructor:** DatabaseConnector()
- **Method:** void connect() throws SQLException, Overrides to call validate() and throw a java.sql.SQLException.

4. Concrete Class: CloudConnector a subclass BaseConnector

- **Constructor:** CloudConnector()
- **Method:** void connect() throws IOException, Overrides to call validate() and throw a java.io.IOException.

Task 2: The main class

Write a demonstration class with a main method:

- create a DatabaseConnector object
- create a CloudConnector object
- Store these objects in an array called systems.
- Write a try-catch block that tries to connect both systems and handle SQLException, IOException, and a general Exception.

Expected Output:

```
Validating configuration for: SQL_DB
[DB ERROR]: Connection timeout: Database server unreachable.

Validating configuration for: S3_BUCKET
[NETWORK ERROR]: Network error: Could not reach cloud storage.
```

2. [10 points] **file processing system:**

You need to write a **file processing system** that reads integers from a file, performs division, and checks for validity. However, various **exceptions** may occur, and you must handle them in your solution. These include:

Custom Exception

- **NegativeNumberException** – Thrown when a number in the file is negative.

Other Exceptions to Handle

- **FileNotFoundException** – Thrown when the input file is missing.
- **NumberFormatException** – Thrown when the file contains non-numeric data.
- **ArithmeticException** – Thrown when attempting division by zero.
- **IOException** – General file reading issues.
- **Finally Clause Requirement** – The program **must close the file properly**, even if an exception occurs.

a) Build a method called "public static void process_data(String data)", which will try to convert the data into an integer num and compute 100/num. It should handle these exceptions: **NegativeNumberException, NumberFormatException, and ArithmeticException.**

b) Build the main method that opens the numbers.txt file, read the data inside line by line and call process_data method for each line. It should handle these exceptions: **NegativeNumberException and IOException.**

Sample output:

```
Enter the filename: numbers.txt
Processed: 25 → Result: 4
Processed: 10 → Result: 10
Math Error: Division by zero is not allowed.
Custom Exception Caught: Negative number found: -5
Data Error: Invalid number format → abc
File closed successfully.
Execution completed (finally block executed).
```

3. [10 points] **Exception throwing**

TestScores
- scores : double[ ]
+ TestScores(s : double[ ]) + getAverage() : double

a) Write a class named **TestScores01**. The class constructor should accept an array of test scores as its argument.
The class should also have a **getAverage()** method that returns the average of the test scores.

b) Demonstrate the class in a program. Use these arrays:
double[] badScores = {97.5, -66.7, 88.0, 101.0, 99.0 };
double[] goodScores = {97.5, 66.7, 88.0, 100.0, 99.0 };

c) Modify the TestScores01 class so that if any test score in the array is negative or greater than 100, the class should not accept these scores and should throw an IllegalArgumentException.

d) Demonstrate the class again and insure that you get the following output:

```
Invalid score found.
Element: 1 Score: -66.7
The average of the good scores is 90.24
```

e) Write an exception class named **InvalidTestScore**.

InvalidTestScore
+InvalidTestScore() +InvalidTestScore(int element, double score)

f) Modify the class you wrote in the previous problem and call it **TestScores02** so that it throws an **InvalidTestScore exception** if any of the test scores in the array are invalid.

g) The output of the demonstration program should be exactly the same.

4. [10 points] **Month Class Exceptions**

a) Write a Month class that holds information about the month according to the UML diagram given.
-The no-arguments constructor Month() should set the month to January.
-The third Constructor takes the month parameter by name: January, February, …
It should work for lower and upper case.

Month
- monthNumber : int
+ Month() + Month(n : int) + Month(name : String) + setMonthNumber(m : int) : void + getMonthNumber() : int + getMonthName() : String + toString() : String + equals(month2 : Month) : boolean + greaterThan(month2 : Month) : boolean + lessThan(month2 : Month) : boolean

b) Write exception classes for the following error conditions:

- **InvalidMonthNumberException** class: A number less than 1 or greater than 12 is given for the month number.
- **InvalidMonthNameException** class: An invalid string is given for the name of the month.

c) Modify the Month class so it throws the appropriate exception when either of these errors occurs.

d) Demonstrate the classes in a program. First, use the no-argument Constructor. Then use a loop and set the month number to the values 0 through 13. The output should be:

```
Using No-argument constructor: Month 1 is January

Error - Invalid number given for the month: 0
Month 1 is January
Month 2 is February
Month 3 is March
Month 4 is April
Month 5 is May
Month 6 is June
Month 7 is July
Month 8 is August
Month 9 is September
Month 10 is October
Month 11 is November
Month 12 is December
Error - Invalid number given for the month: 13
```